Source Code Management using GitHub SOP

Cognizant

Table of Contents

1. Source Code Management using GitHub.....3
1.1 Description.....3
1.2 Architecture Diagram.....3
1.3 Lab Steps.....4
1.4 Troubleshooting.....11
1.5 Supporting References.....11

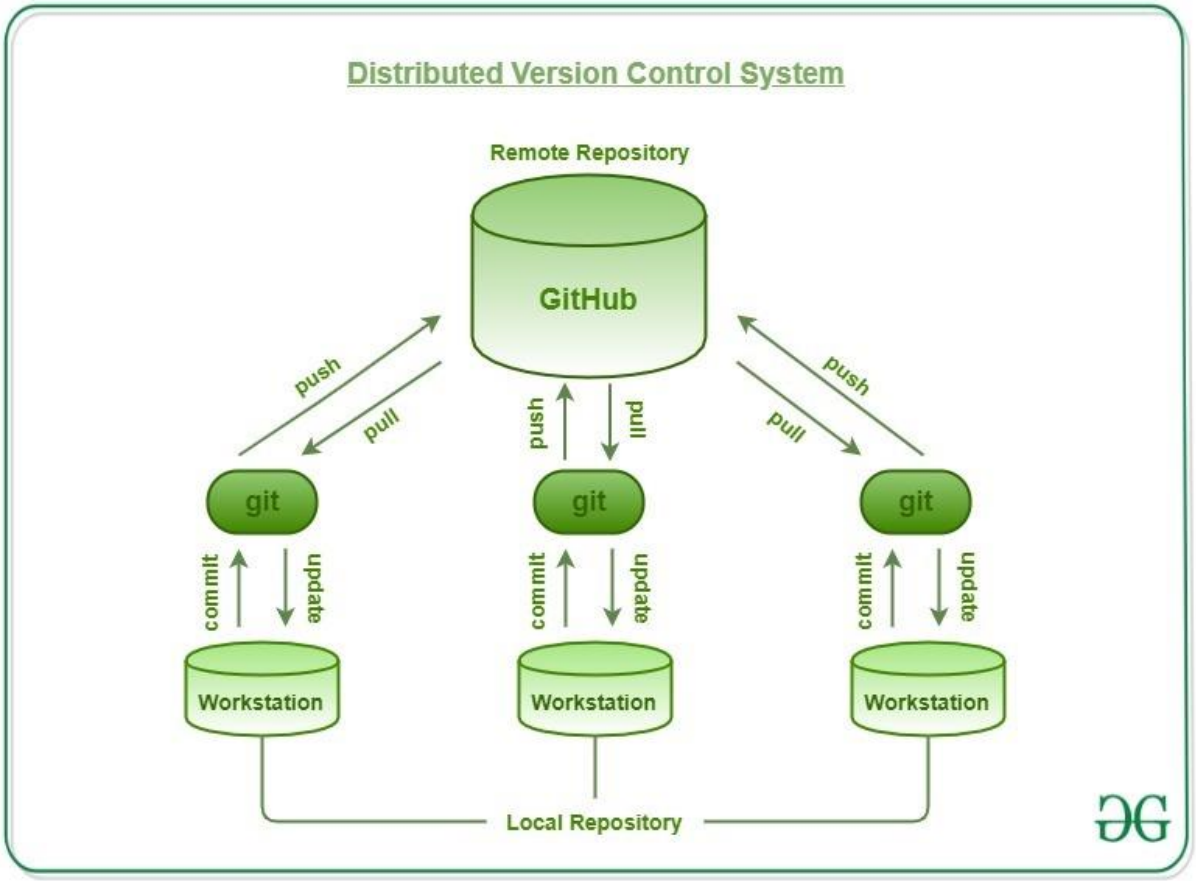
1. Source Code Management using GitHub

1.1 Description

In this lab exercise, we will help users to create repository and manage code.

1.2 Architecture Diagram

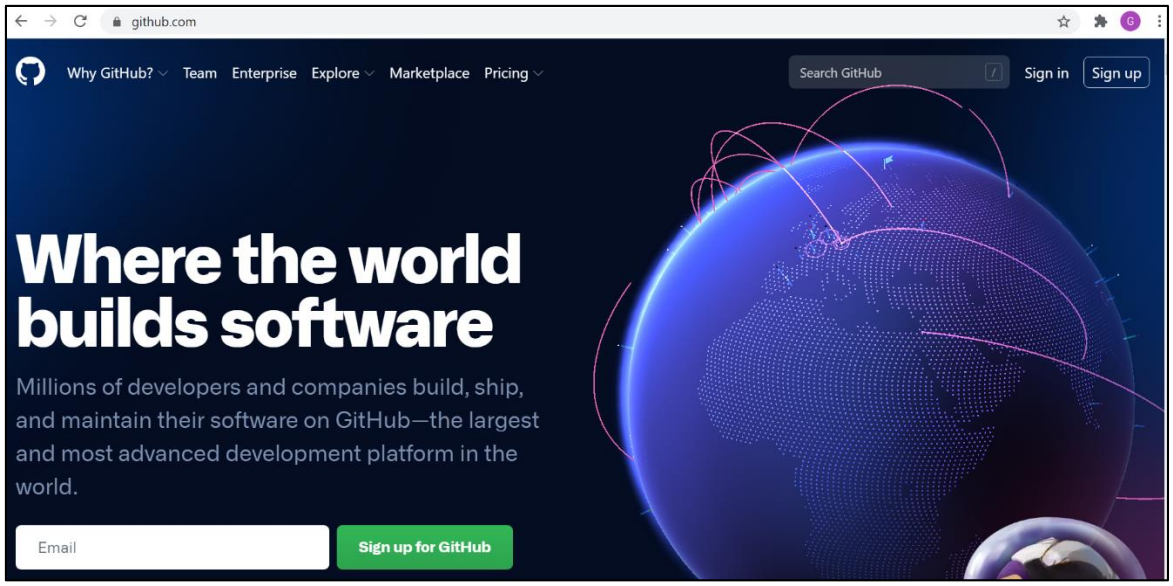
The diagram below displays a visual representation of the underlying application architecture:



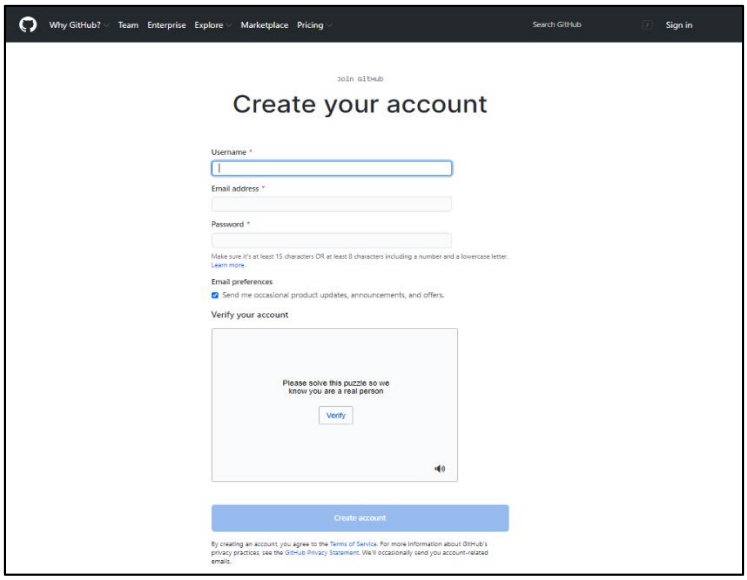
1.3 Lab Steps

Follow the steps outlined below to achieve the objectives of this lab exercise:

1. Installing Software:
   a. Sign up for GitHub, creating a username and password.
   GitHub URL: https://github.com/

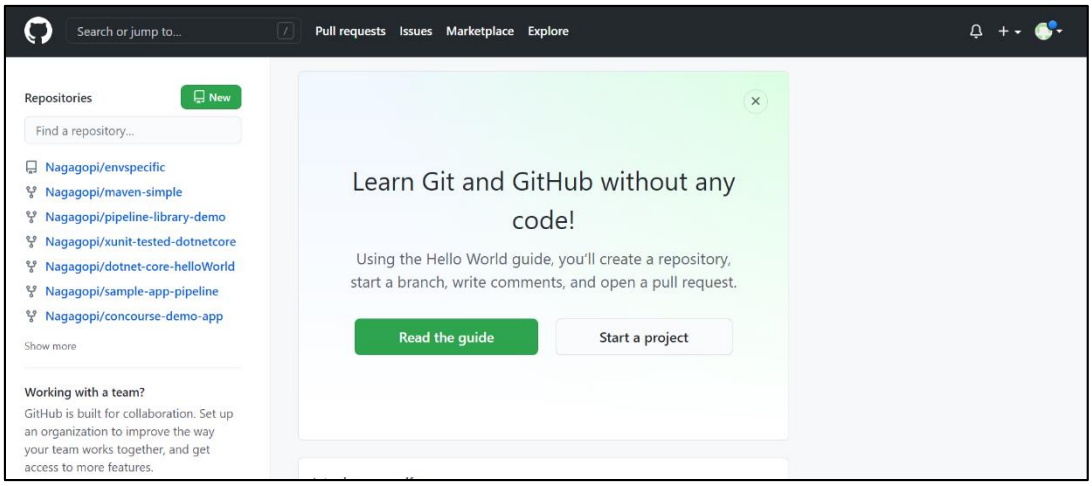


   b. Fill all the necessary details such as Username, Password and Email address and then click on **Create Account**.

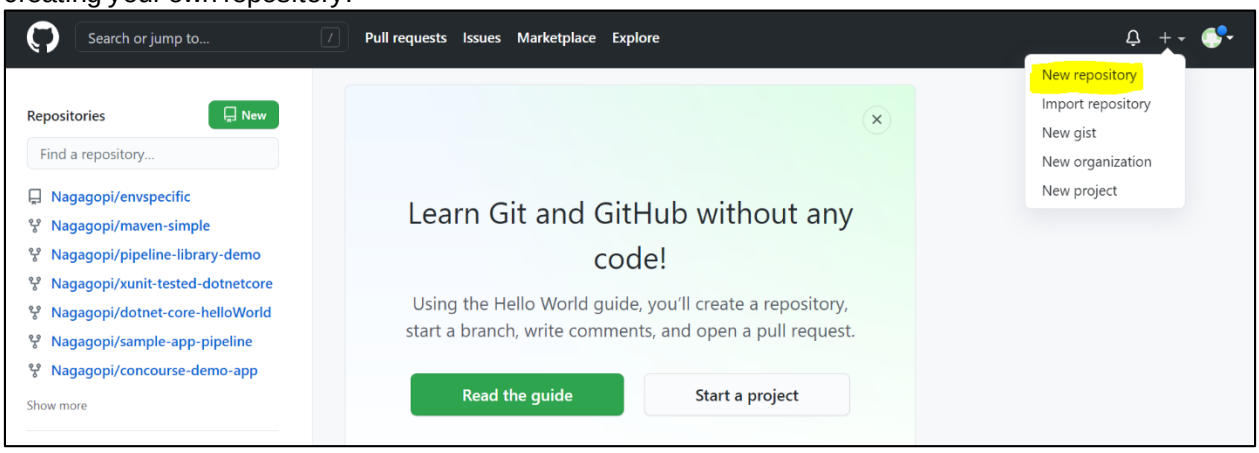


   c. Download GitHub for Windows. GitHub for Windows is a GUI client that enables you to work on files offline before making them public: http://windows.github.com/
   d. Enter in the username/password that you used for GitHub.

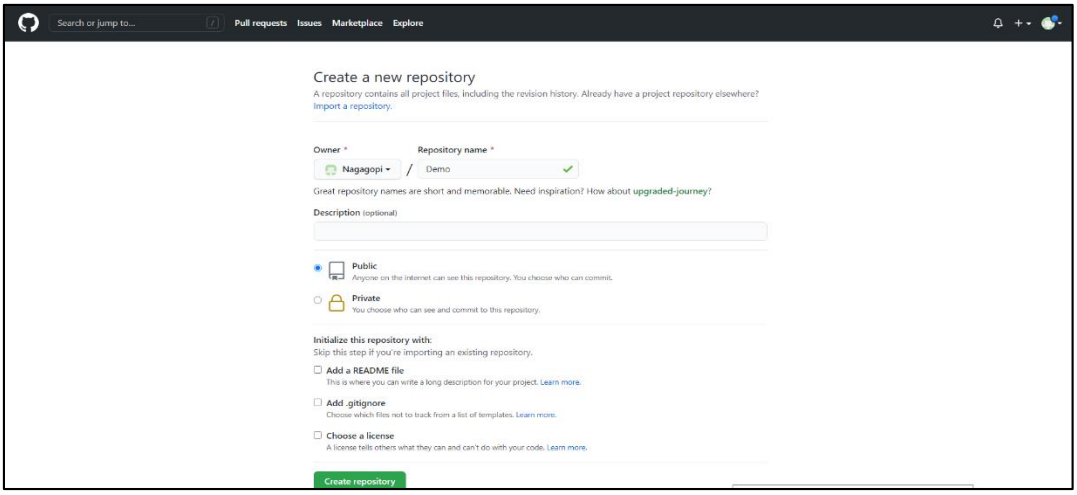
2. Setting up a GitHub Repository:
   a. Log in to your GitHub.com account



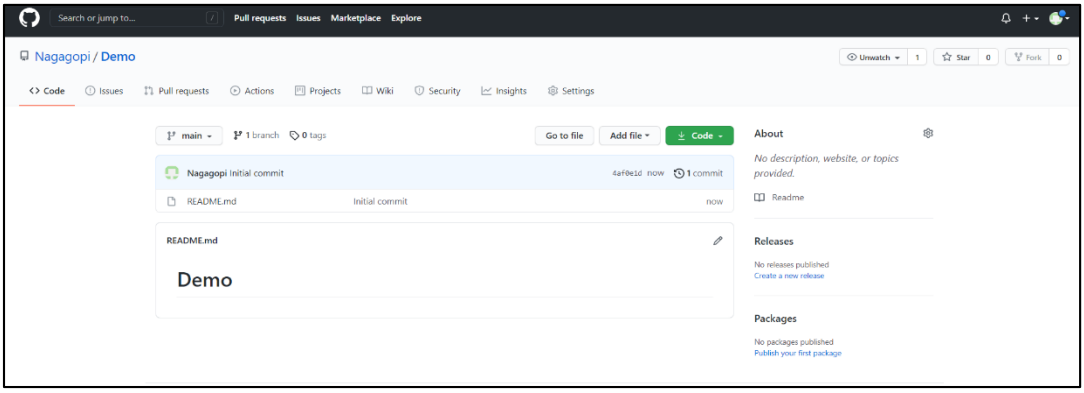
   b. Once logged in you will be brought to a front page (dashboard) where you can get started on creating your own repository.



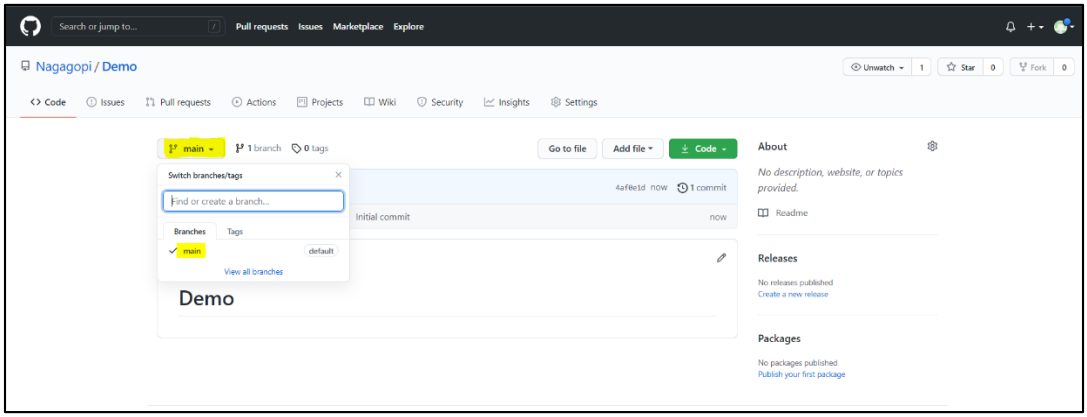
c. Fill in the following mandatory details to create a new repository:
   i. Owner
   ii. Repository Name
   iii. Public/Private



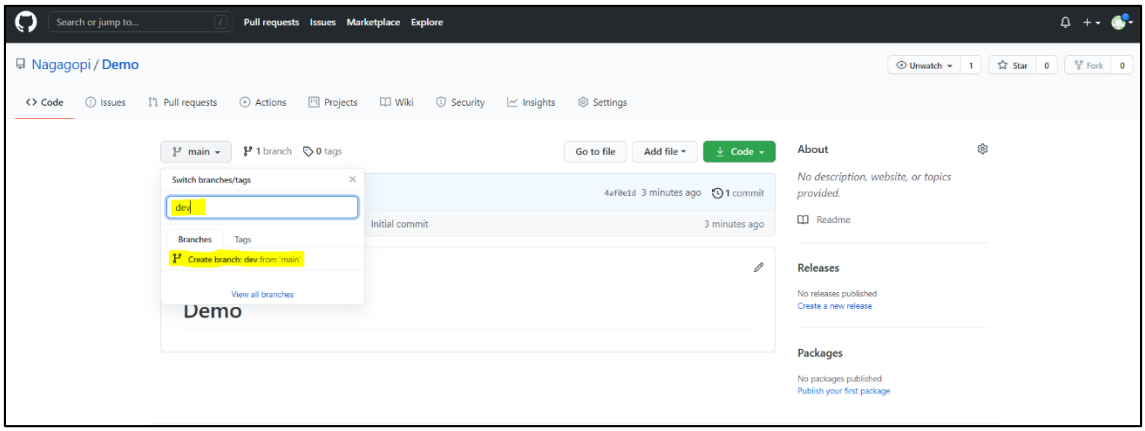
Repository has been created.



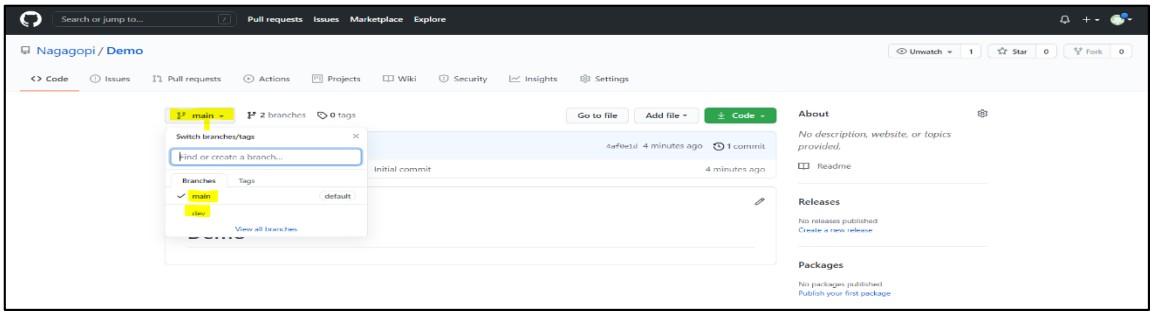
3. GitHub Workflow:
   The GitHub flow has different steps, each with distinct benefits when implemented:
   a. **Create a branch**: Topic branches created from the canonical deployment branch (usually master) allow teams to contribute to many parallel efforts. Short-lived topic branches, in particular, keep teams focused and results in quick ships.
      i. By Default, the main branch will be created when we created the repository.



   b. If needed new branch can also be created:

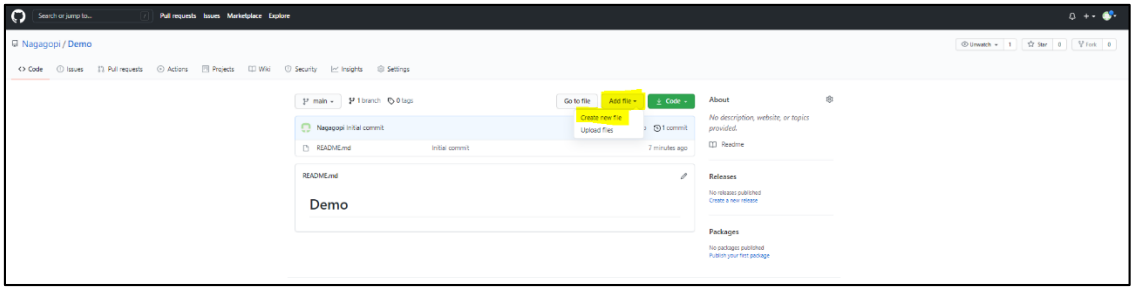


New branch **dev** has been created successfully.

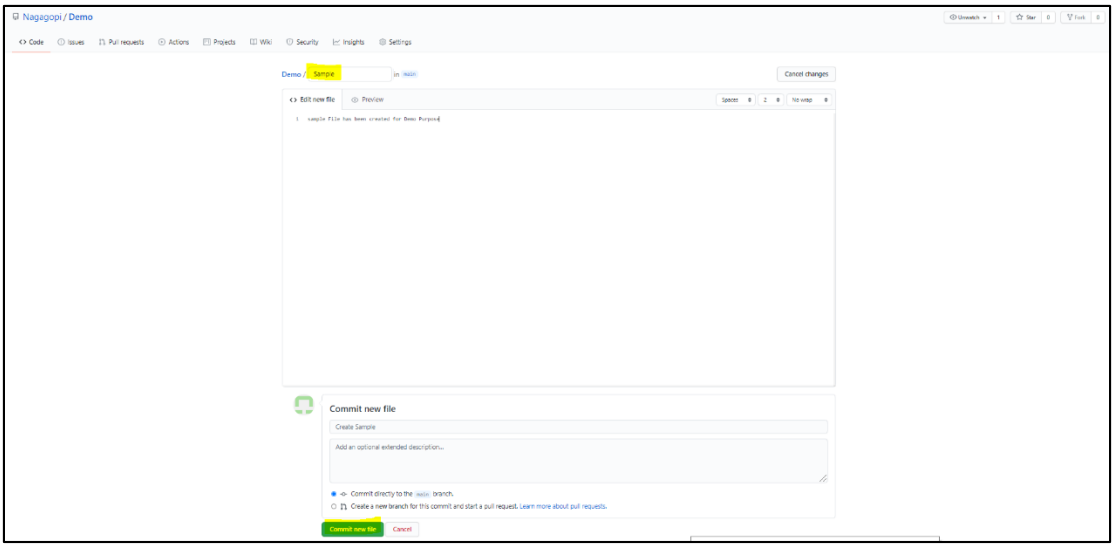


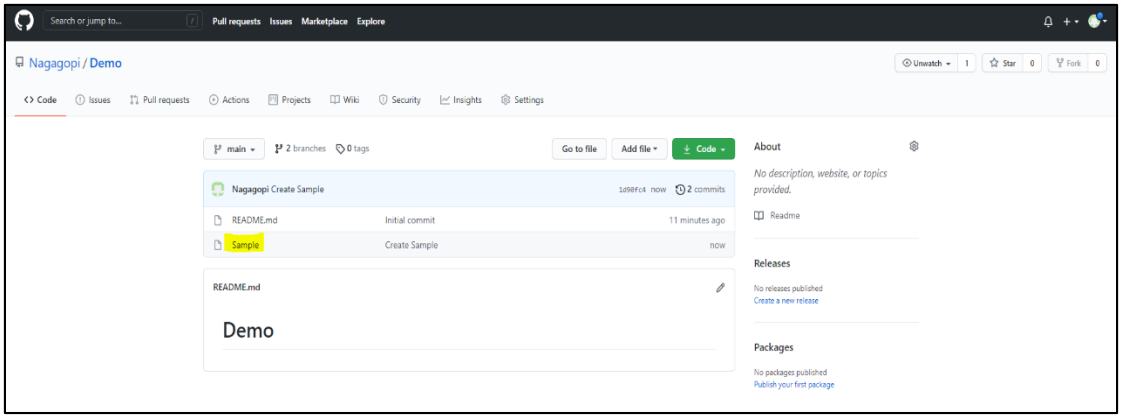
c. **Add commits**: Snapshots of development efforts within a branch create safe, revertible points in the project's history.

i. In the repository new files can be created or already existing files can be uploaded. Click on **Add file** > Select Option based on requirement.
Here we are creating a new file.



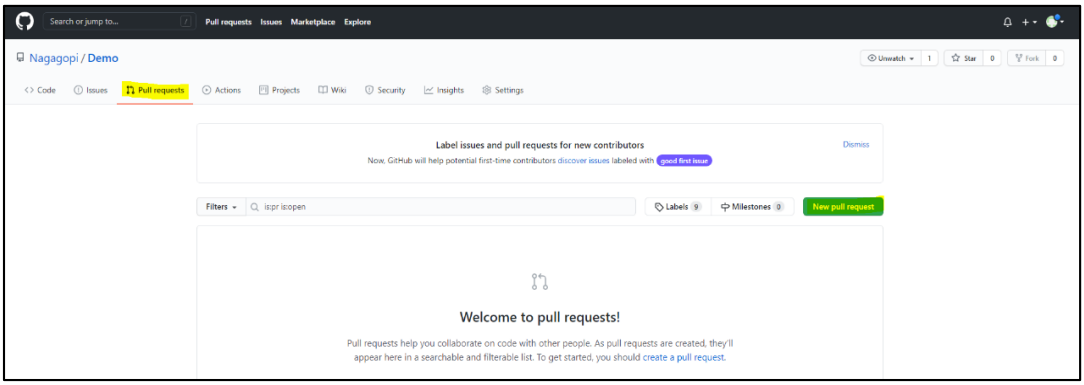
New file has been created and committed:



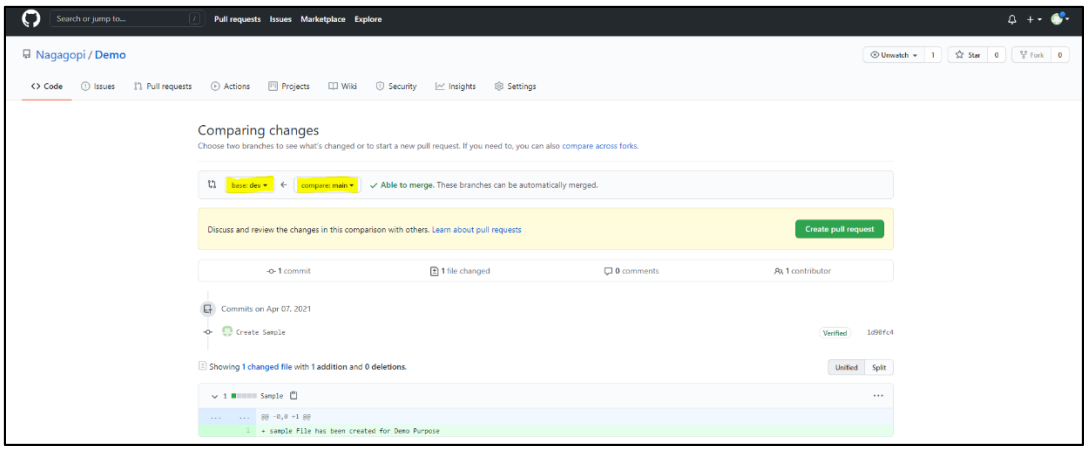


d. **Open a pull request**: Pull requests publicize a project's ongoing efforts and set the tone for a transparent development process.
When you open a pull request, you are proposing your changes and requesting that someone review and pull in your contribution and merge them into their branch. Pull requests show diffs, or differences, of the content from both branches.

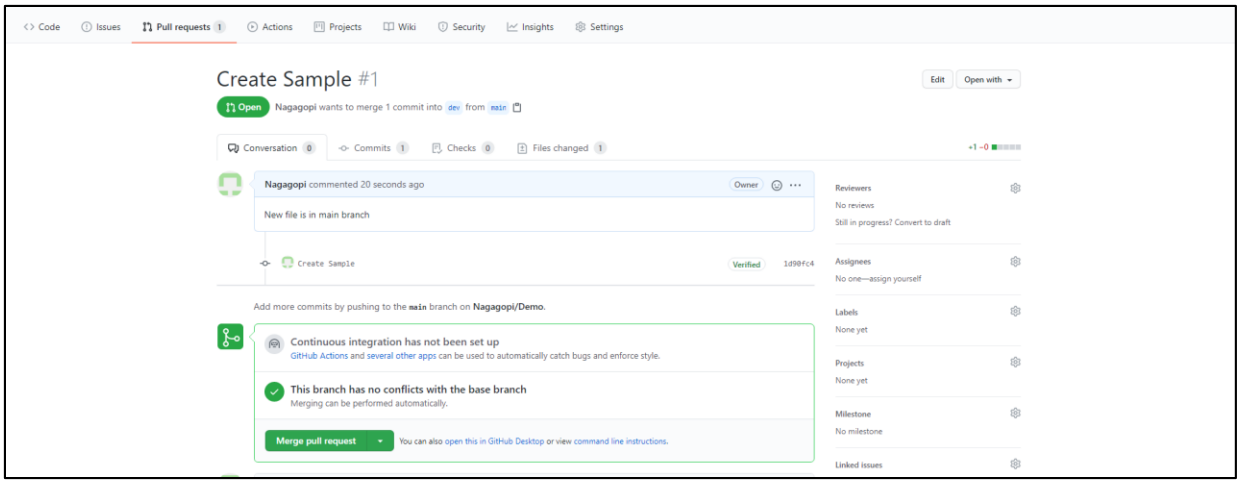
i. Create a pull request in GitHub as shown below:
Click on **New Pull request**:



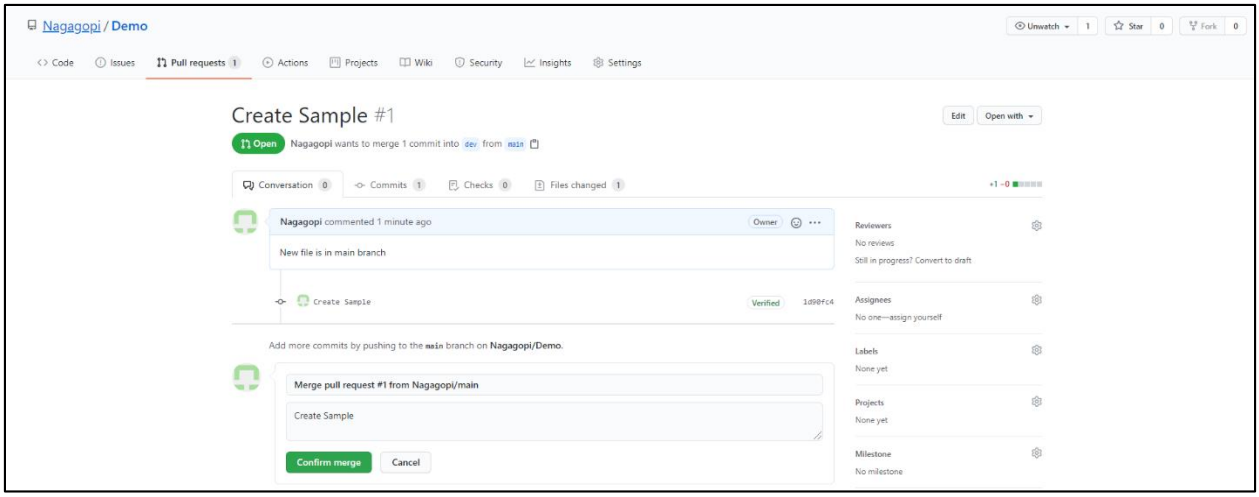
ii. Select the base branch and compare branch.
Earlier we committed a new file Sample in the Main branch and it is not committed to dev branch so it is displaying below the difference between two branches.



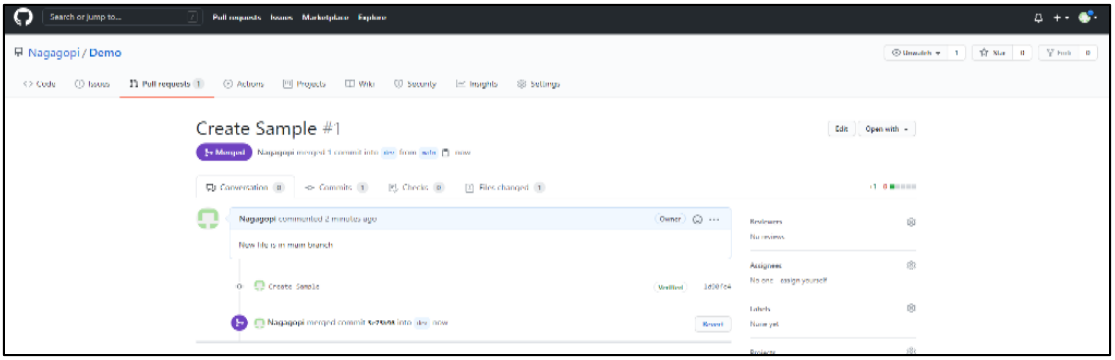
e. **Discuss and review code**: Teams participate in code reviews by commenting, testing, and reviewing open pull requests. Code review is at the core of an open and participatory culture.



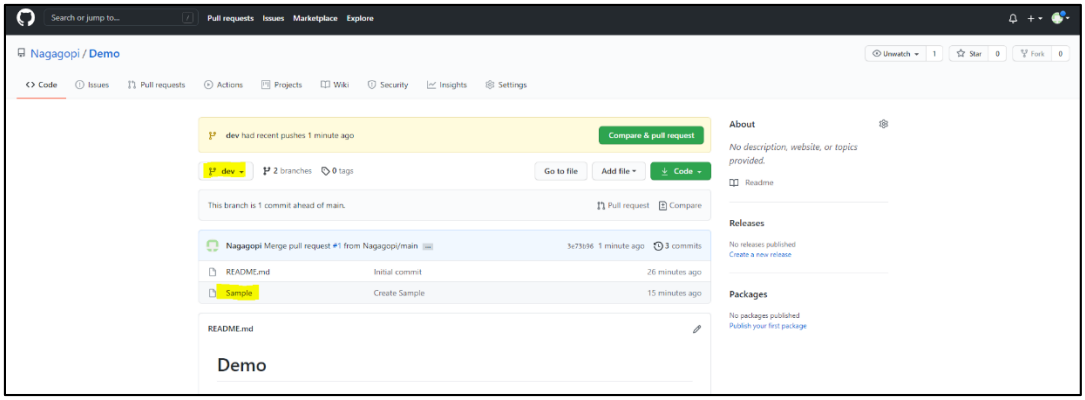
f. **Merge**: Upon clicking merge, GitHub automatically performs the equivalent of a local **git merge** operation. GitHub also keeps the entire branch development history on the merged pull request.



i. After clicking on **Confirm merge**, the code has been merged into the dev branch.



Because of the merge the Sample file has been copied to dev branch as well:



g. **Deploy**: Teams can choose the best release cycles or incorporate continuous integration tools and operate with the assurance that code on the deployment branch has gone through a robust workflow.

1.4 Troubleshooting

S. No	Problem	Solution
1	Automatic build of code for a compiled project fails.	Remove the autobuild step from your code scanning workflow and add specific build steps.
2	No source code was seen during the build	Scanning workflow is analyzing a compiled language (C, C++, C#, or Java), but the code was not compiled. By default, the CodeQL analysis workflow contains an autobuild step, however, this step represents a best effort process, and may not succeed in building your code, depending on your specific build environment. Compilation may also fail if you have removed the autobuild step and did not include build steps manually.

1.5 Supporting References

Refer the below link for additional information:

1. https://support.github.com/